

# DeepONet: Deep Operator Network

Mathematical Formulation and PyTorch Reference Implementation

## 1 Introduction

DeepONet, short for *Deep Operator Network*, is a neural operator architecture introduced for learning nonlinear mappings between function spaces. The central objective is fundamentally different from conventional finite-dimensional regression.

A conventional neural network approximates a mapping

$$f : \mathbb{R}^n \rightarrow \mathbb{R}^m. \quad (1)$$

An operator-learning problem instead seeks to approximate

$$\boxed{\mathcal{G} : \mathcal{A} \rightarrow \mathcal{U}} \quad (2)$$

where  $\mathcal{A}$  and  $\mathcal{U}$  are spaces of functions.

For an input function  $u$ , the output of the operator is another function

$$v = \mathcal{G}(u). \quad (3)$$

DeepONet approximates this operator using two neural networks:

1. the **branch network**, which encodes the input function;
2. the **trunk network**, which encodes the location at which the output function is evaluated.

This branch–trunk decomposition is the defining structural feature of DeepONet. The original DeepONet work demonstrates its application to explicit operators such as integrals and fractional operators as well as implicit operators associated with differential equations. [1]

## 2 Operator Learning Problem

Let

$$\mathcal{G} : \mathcal{A} \rightarrow \mathcal{U} \quad (4)$$

be a nonlinear operator.

Given an input function

$$u : \Omega \rightarrow \mathbb{R}, \quad (5)$$

the desired output is

$$v(y) = \mathcal{G}(u)(y), \quad y \in \Omega'. \quad (6)$$

The important point is that the output is itself a function.

For example, consider a parameterized PDE

$$\mathcal{L}_a[v](x) = f(x), \quad (7)$$

where  $a(x)$  is an input coefficient field.

The corresponding solution operator is

$$\boxed{\mathcal{G} : a(\cdot) \mapsto v(\cdot)}. \quad (8)$$

DeepONet attempts to learn this entire mapping rather than training a separate neural network for each particular input function.

### 3 Sensor Representation of the Input Function

A neural network cannot directly process an arbitrary infinite-dimensional function. DeepONet therefore represents the input function through its values at a finite collection of sensors.

Let

$$\{x_1, x_2, \dots, x_m\} \subset \Omega \quad (9)$$

be sensor locations.

The input function is represented by

$$\mathbf{u} = \begin{bmatrix} u(x_1) \\ u(x_2) \\ \vdots \\ u(x_m) \end{bmatrix} \in \mathbb{R}^m. \quad (10)$$

Define the sensor projection

$$\mathcal{D}(u) = [u(x_1), \dots, u(x_m)]^T. \quad (11)$$

The branch network receives this finite-dimensional representation.

### 4 Branch Network

The branch network is a neural mapping

$$\mathcal{B}_{\theta_b} : \mathbb{R}^m \rightarrow \mathbb{R}^p. \quad (12)$$

Given the sampled input function,

$$\mathbf{u} = \mathcal{D}(u), \quad (13)$$

the branch network produces

$$\mathbf{b} = \mathcal{B}_{\theta_b}(\mathbf{u}). \quad (14)$$

Writing its components explicitly,

$$\mathbf{b} = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_p \end{bmatrix}. \quad (15)$$

The vector  $\mathbf{b}$  is a latent representation of the input function.

## 4.1 Fully Connected Branch Network

For a fully connected branch network,

$$h_b^{(0)} = \mathbf{u}. \quad (16)$$

For layer  $\ell$ ,

$$h_b^{(\ell+1)} = \sigma \left( W_b^{(\ell)} h_b^{(\ell)} + b_b^{(\ell)} \right). \quad (17)$$

The final branch representation is

$$\boxed{\mathbf{b} = W_b^{(L)} h_b^{(L)} + b_b^{(L)}}. \quad (18)$$

Thus,

$$b_k = \mathcal{B}_k(u(x_1), \dots, u(x_m)). \quad (19)$$

## 5 Trunk Network

The trunk network receives the location at which the output is to be evaluated.

Let

$$y \in \Omega' \quad (20)$$

denote an output coordinate.

The trunk network is

$$\mathcal{T}_{\theta_t} : \Omega' \rightarrow \mathbb{R}^p. \quad (21)$$

Thus,

$$\mathbf{t}(y) = \mathcal{T}_{\theta_t}(y) \quad (22)$$

with

$$\mathbf{t}(y) = \begin{bmatrix} t_1(y) \\ t_2(y) \\ \vdots \\ t_p(y) \end{bmatrix}. \quad (23)$$

Unlike the branch network, the trunk network does not encode the input function. It encodes the output location.

### 5.1 Fully Connected Trunk Network

For a multilayer perceptron,

$$h_t^{(0)} = y. \quad (24)$$

The hidden layers are

$$h_t^{(\ell+1)} = \sigma \left( W_t^{(\ell)} h_t^{(\ell)} + b_t^{(\ell)} \right). \quad (25)$$

The final representation is

$$\boxed{\mathbf{t}(y) = W_t^{(L)} h_t^{(L)} + b_t^{(L)}}. \quad (26)$$

## 6 Branch–Trunk Combination

The defining operation of DeepONet is the inner product between the branch and trunk representations.

The predicted output is

$$\mathcal{G}(\hat{u})(y) = \sum_{k=1}^p b_k(u) t_k(y) + b_0. \quad (27)$$

Equivalently,

$$\hat{v}(y) = \mathbf{b}(u)^T \mathbf{t}(y) + b_0. \quad (28)$$

This is the fundamental DeepONet equation.

The branch network determines coefficients

$$b_1(u), \dots, b_p(u) \quad (29)$$

depending on the input function, while the trunk network provides basis-like functions

$$t_1(y), \dots, t_p(y) \quad (30)$$

depending on the output location.

Consequently,

$$\hat{v}(y) = \sum_{k=1}^p b_k(u) t_k(y). \quad (31)$$

## 7 Interpretation as a Learned Basis Expansion

The DeepONet representation can be interpreted as a learned expansion of the output function.

For a fixed input  $u$ , define

$$c_k(u) = b_k(u). \quad (32)$$

Then

$$\hat{v}(y) = \sum_{k=1}^p c_k(u) t_k(y). \quad (33)$$

The trunk functions act as learned basis functions while the branch network produces input-dependent coefficients.

This resembles classical reduced-order representations such as

$$v(y) \approx \sum_{k=1}^p c_k \phi_k(y), \quad (34)$$

but with a crucial difference:

$$c_k = \mathcal{B}_k(u) \quad (35)$$

is learned directly from the input function rather than computed through a fixed projection procedure.

## 8 General DeepONet Operator

Combining the branch and trunk networks gives

$$\mathcal{G}_\theta(u)(y) = \sum_{k=1}^p \mathcal{B}_k(\mathcal{D}(u)) \mathcal{T}_k(y) + b_0. \quad (36)$$

In vector notation,

$$\mathcal{G}_\theta(u)(y) = \mathcal{B}_{\theta_b}(\mathcal{D}(u))^T \mathcal{T}_{\theta_t}(y) + b_0. \quad (37)$$

This equation describes the complete basic DeepONet architecture.

## 9 Universal Approximation Perspective

The theoretical motivation for DeepONet comes from universal approximation results for non-linear operators.

Consider a continuous nonlinear operator

$$\mathcal{G} : \mathcal{V} \rightarrow \mathcal{U}. \quad (38)$$

Under suitable assumptions, the operator can be approximated by a finite separable representation of the form

$$\mathcal{G}(u)(y) \approx \sum_{k=1}^p c_k(u) \phi_k(y). \quad (39)$$

DeepONet replaces the unknown coefficient functions  $c_k(u)$  and basis functions  $\phi_k(y)$  with neural networks:

$$c_k(u) \approx \mathcal{B}_k(\mathcal{D}(u)), \quad (40)$$

$$\phi_k(y) \approx \mathcal{T}_k(y). \quad (41)$$

Therefore,

$$\mathcal{G}(u)(y) \approx \sum_{k=1}^p \mathcal{B}_k(\mathcal{D}(u)) \mathcal{T}_k(y). \quad (42)$$

This provides the mathematical foundation for the branch–trunk architecture. The original work develops DeepONet from the universal approximation theorem for operators. [1]

## 10 Example: Integral Operator

Consider the integral operator

$$\mathcal{G}(u)(y) = \int_{\Omega} K(y, x) u(x) dx. \quad (43)$$

This is an operator mapping a function  $u(x)$  to another function of  $y$ .

Suppose that the kernel admits an approximate separable representation

$$K(y, x) \approx \sum_{k=1}^p a_k(y) b_k(x). \quad (44)$$

Then

$$\mathcal{G}(u)(y) \approx \int_{\Omega} \sum_{k=1}^p a_k(y) b_k(x) u(x) dx \quad (45)$$

$$= \sum_{k=1}^p a_k(y) \int_{\Omega} b_k(x) u(x) dx. \quad (46)$$

Define

$$c_k(u) = \int_{\Omega} b_k(x) u(x) dx. \quad (47)$$

Then

$$\mathcal{G}(u)(y) \approx \sum_{k=1}^p c_k(u) a_k(y). \quad (48)$$

DeepONet learns both components:

$$c_k(u) \approx \mathcal{B}_k(u), \quad (49)$$

$$a_k(y) \approx \mathcal{T}_k(y). \quad (50)$$

Thus,

$$\boxed{\hat{\mathcal{G}}(u)(y) = \sum_{k=1}^p \mathcal{B}_k(u) \mathcal{T}_k(y)}. \quad (51)$$

## 11 Example: PDE Solution Operator

Consider the parameterized PDE

$$\mathcal{L}_a[u](x) = f(x), \quad x \in \Omega, \quad (52)$$

with appropriate boundary conditions.

The coefficient field  $a(x)$  may vary between different problem instances.

The corresponding solution operator is

$$\mathcal{G}(a) = u. \quad (53)$$

DeepONet approximates

$$\hat{u}(y) = \sum_{k=1}^p \mathcal{B}_k(a(x_1), \dots, a(x_m)) \mathcal{T}_k(y). \quad (54)$$

Once trained, the same model can evaluate the solution at arbitrary locations  $y$  supplied to the trunk network.

## 12 Multiple Output Locations

Suppose the solution is required at

$$y_1, \dots, y_n. \quad (55)$$

For each location,

$$\hat{u}(y_j) = \mathbf{b}(u)^T \mathbf{t}(y_j). \quad (56)$$

Stacking the trunk outputs gives

$$T = \begin{bmatrix} \mathbf{t}(y_1)^T \\ \mathbf{t}(y_2)^T \\ \vdots \\ \mathbf{t}(y_n)^T \end{bmatrix} \in \mathbb{R}^{n \times p}. \quad (57)$$

Then

$$\boxed{\hat{\mathbf{u}} = T\mathbf{b} + b_0\mathbf{1}}. \quad (58)$$

This matrix formulation is particularly useful for efficient evaluation of an entire output field.

## 13 Training Objective

Suppose the training dataset contains

$$\left\{ u^{(i)}, v^{(i)} \right\}_{i=1}^N. \quad (59)$$

The target satisfies

$$v^{(i)} = \mathcal{G} \left( u^{(i)} \right). \quad (60)$$

For output locations  $y_j$ , the prediction is

$$\hat{v}^{(i)}(y_j) = \mathcal{G}_\theta \left( u^{(i)} \right) (y_j). \quad (61)$$

A standard mean-squared loss is

$$\boxed{\mathcal{L}_{\text{data}} = \frac{1}{Nn} \sum_{i=1}^N \sum_{j=1}^n \left| \hat{v}^{(i)}(y_j) - v^{(i)}(y_j) \right|^2}. \quad (62)$$

A relative  $L_2$  loss can instead be written as

$$\mathcal{L}_{\text{rel}} = \frac{1}{N} \sum_{i=1}^N \frac{\|\hat{v}^{(i)} - v^{(i)}\|_2}{\|v^{(i)}\|_2 + \epsilon}. \quad (63)$$

## 14 Physics-Informed DeepONet

DeepONet can also be trained without requiring complete paired input–output datasets when the governing differential equation is known.

Let

$$u_\theta(y) = \mathcal{G}_\theta(a)(y). \quad (64)$$

Suppose the PDE is

$$\mathcal{L}[u(y)] = f(y). \quad (65)$$

The neural operator residual is

$$r_\theta(y) = \mathcal{L}[u_\theta(y)] - f(y). \quad (66)$$

The physics loss is

$$\mathcal{L}_{\text{PDE}} = \frac{1}{N_c} \sum_{j=1}^{N_c} |r_\theta(y_j)|^2. \quad (67)$$

Boundary conditions can be enforced using

$$\mathcal{L}_{\text{BC}} = \frac{1}{N_b} \sum_{j=1}^{N_b} |u_\theta(y_j) - g(y_j)|^2. \quad (68)$$

The complete physics-informed objective is

$$\mathcal{L} = \lambda_{\text{data}} \mathcal{L}_{\text{data}} + \lambda_{\text{PDE}} \mathcal{L}_{\text{PDE}} + \lambda_{\text{BC}} \mathcal{L}_{\text{BC}}. \quad (69)$$

Automatic differentiation through the trunk network permits spatial and temporal derivatives of the predicted output to be calculated directly. Physics-informed DeepONets have been demonstrated for parametric PDE solution operators. [2]

## 15 Computational Structure

For  $m$  input sensors,  $n$  output locations, and latent dimension  $p$ , the branch network is evaluated once for a given input function:

$$\mathbf{b} = \mathcal{B}(\mathbf{u}). \quad (70)$$

The trunk network evaluates the output coordinates:

$$T = \mathcal{T}(Y), \quad (71)$$

where

$$Y = \{y_1, \dots, y_n\}. \quad (72)$$

The output is obtained through

$$\hat{\mathbf{v}} = T\mathbf{b}. \quad (73)$$

The final combination is therefore inexpensive once the two latent representations have been calculated.



## 16 Strengths and Limitations

DeepONet has several important properties.

### 16.1 Strengths

1. **Operator-level learning:** the model learns a mapping between functions rather than merely between fixed-dimensional vectors.
2. **Arbitrary output evaluation:** the trunk network accepts output coordinates directly, allowing the learned function to be evaluated at different locations.
3. **Flexible architectures:** the branch network can be implemented using fully connected networks, convolutional networks, recurrent networks, or other architectures.
4. **Compatibility with physics:** automatic differentiation enables PDE residuals and boundary conditions to be incorporated into training.
5. **Sensor-based representation:** the input function can be represented using scattered sensor measurements rather than requiring a regular grid.

### 16.2 Limitations

The classical DeepONet also has limitations.

The input function must first be projected onto a finite set of sensors:

$$u \longrightarrow \mathcal{D}(u). \quad (74)$$

Therefore, the choice and number of sensors can strongly affect approximation quality. Furthermore, the output representation has the separable form

$$\hat{u}(y) = \sum_{k=1}^p b_k(u) t_k(y). \quad (75)$$

A small value of  $p$  may therefore limit the expressive capacity of the learned operator.

Increasing  $p$  increases representational capacity but also increases the number of parameters and computational cost.

## 17 DeepONet Compared with Standard Neural Networks

A conventional neural network learns

$$\mathbf{x} \mapsto \mathbf{y}. \quad (76)$$

DeepONet instead learns

$$u(\cdot) \mapsto v(\cdot). \quad (77)$$

More explicitly,

$$\boxed{\mathbf{x} \rightarrow \mathbf{y}} \quad (78)$$

versus

$$\boxed{u(\cdot) \rightarrow \mathcal{G}(u)(\cdot)}. \quad (79)$$

The branch network encodes the input function while the trunk network provides the dependence of the output on its evaluation coordinate.

This separation is the defining mechanism that allows DeepONet to learn families of input–output functions.

## 18 PyTorch Implementation

The following implementation directly realizes the classical branch–trunk DeepONet formulation.

```
1
2 import torch
3 import torch.nn as nn
4
5
6 # =====
7 # Generic MLP
8 # =====
9
10 class MLP(nn.Module):
11
12     def __init__(
13         self,
14         input_dim,
15         hidden_dim,
16         output_dim,
17         depth=3
18     ):
19         super().__init__()
20
21         layers = []
22
23         layers.append(
24             nn.Linear(
25                 input_dim,
26                 hidden_dim
27             )
28         )
29
30         layers.append(
31             nn.GELU()
32         )
33
34         for _ in range(depth - 1):
35
36             layers.append(
37                 nn.Linear(
38                     hidden_dim,
39                     hidden_dim
40                 )
41             )
42
43             layers.append(
44                 nn.GELU()
45             )
46
47         layers.append(
48             nn.Linear(
```

```

49         hidden_dim,
50         output_dim
51     )
52 )
53
54 self.network = nn.Sequential(
55     *layers
56 )
57
58 def forward(self, x):
59
60     return self.network(x)
61
62
63 # =====
64 # Branch Network
65 # =====
66
67 class BranchNet(nn.Module):
68
69     """
70     Encodes the input function evaluated at fixed
71     sensor locations.
72     """
73
74     def __init__(
75         self,
76         num_sensors,
77         latent_dim,
78         hidden_dim=128,
79         depth=3
80     ):
81         super().__init__()
82
83         self.network = MLP(
84             input_dim=num_sensors,
85             hidden_dim=hidden_dim,
86             output_dim=latent_dim,
87             depth=depth
88         )
89
90     def forward(self, x):
91
92         # x:
93         # [batch, num_sensors]
94
95         return self.network(x)
96
97
98 # =====
99 # Trunk Network
100 # =====
101
102 class TrunkNet(nn.Module):
103
104     """
105     Encodes output coordinates.
106     """

```

```

107
108     def __init__(
109         self,
110         coordinate_dim,
111         latent_dim,
112         hidden_dim=128,
113         depth=3
114     ):
115         super().__init__()
116
117         self.network = MLP(
118             input_dim=coordinate_dim,
119             hidden_dim=hidden_dim,
120             output_dim=latent_dim,
121             depth=depth
122         )
123
124     def forward(self, y):
125
126         # y:
127         # [num_points, coordinate_dim]
128         #
129         # or
130         #
131         # [batch, num_points, coordinate_dim]
132
133         return self.network(y)
134
135
136 # =====
137 # DeepONet
138 # =====
139
140 class DeepONet(nn.Module):
141
142     def __init__(
143         self,
144         num_sensors,
145         coordinate_dim,
146         latent_dim=128,
147         hidden_dim=128,
148         depth=3
149     ):
150         super().__init__()
151
152         self.branch = BranchNet(
153             num_sensors=num_sensors,
154             latent_dim=latent_dim,
155             hidden_dim=hidden_dim,
156             depth=depth
157         )
158
159         self.trunk = TrunkNet(
160             coordinate_dim=coordinate_dim,
161             latent_dim=latent_dim,
162             hidden_dim=hidden_dim,
163             depth=depth
164         )

```

```

165         # Learnable bias
166         self.bias = nn.Parameter(
167             torch.zeros(1)
168         )
169
170     def forward(
171         self,
172         branch_input,
173         trunk_input
174     ):
175
176         # -----
177         # Branch representation
178         # -----
179
180         b = self.branch(
181             branch_input
182         )
183
184         # b:
185         # [B, p]
186
187         # -----
188         # Trunk representation
189         # -----
190
191         t = self.trunk(
192             trunk_input
193         )
194
195         # If trunk input has no batch dimension:
196         #
197         # t:
198         # [N, p]
199
200         if t.dim() == 2:
201
202             output = torch.einsum(
203                 "bp,np->bn",
204                 b,
205                 t
206             )
207
208         # If trunk input has a batch dimension:
209         #
210         # t:
211         # [B, N, p]
212
213         elif t.dim() == 3:
214
215             output = torch.einsum(
216                 "bp,bnp->bn",
217                 b,
218                 t
219             )
220
221         else:

```

```

223         raise ValueError(
224             "Unexpected trunk tensor dimension."
225         )
226
227     output = output + self.bias
228
229     return output
230
231
232
233 # =====
234 # Relative L2 loss
235 # =====
236
237 def relative_l2(
238     prediction,
239     target,
240     eps=1e-8
241 ):
242
243     numerator = torch.norm(
244         prediction - target
245     )
246
247     denominator = (
248         torch.norm(target)
249         + eps
250     )
251
252     return numerator / denominator
253
254
255 # =====
256 # Example: PDE-style operator learning
257 # =====
258
259 if __name__ == "__main__":
260
261     device = (
262         "cuda"
263         if torch.cuda.is_available()
264         else "cpu"
265     )
266
267     # -----
268     # Problem dimensions
269     # -----
270
271     batch_size = 16
272
273     num_sensors = 100
274
275     coordinate_dim = 2
276
277     num_output_points = 256
278
279     latent_dim = 128
280

```

```

281 # -----
282 # Example input functions
283 # -----
284
285 branch_input = torch.randn(
286     batch_size,
287     num_sensors,
288     device=device
289 )
290
291 # -----
292 # Output coordinates
293 # -----
294
295 trunk_input = torch.rand(
296     num_output_points,
297     coordinate_dim,
298     device=device
299 )
300
301 # -----
302 # Target solution
303 # -----
304
305 target = torch.randn(
306     batch_size,
307     num_output_points,
308     device=device
309 )
310
311 # -----
312 # Model
313 # -----
314
315 model = DeepONet(
316     num_sensors=num_sensors,
317     coordinate_dim=coordinate_dim,
318     latent_dim=latent_dim,
319     hidden_dim=128,
320     depth=3
321 ).to(device)
322
323 # -----
324 # Forward pass
325 # -----
326
327 prediction = model(
328     branch_input,
329     trunk_input
330 )
331
332 # -----
333 # Loss
334 # -----
335
336 loss = relative_l2(
337     prediction,
338     target

```

```

339     )
340
341     print(
342         "Prediction shape:",
343         prediction.shape
344     )
345
346     print(
347         "Loss:",
348         loss.item()
349     )
350
351     % =====
352     \section{Training Loop}
353     % =====
354
355     A standard supervised training procedure is
356
357     \begin{lstlisting}
358
359     optimizer = torch.optim.Adam(
360         model.parameters(),
361         lr=1e-3
362     )
363
364     epochs = 100
365
366     for epoch in range(epochs):
367
368         model.train()
369
370         optimizer.zero_grad()
371
372         prediction = model(
373             branch_input,
374             trunk_input
375         )
376
377         loss = relative_l2(
378             prediction,
379             target
380         )
381
382         loss.backward()
383
384         optimizer.step()
385
386         if epoch % 10 == 0:
387
388             print(
389                 f"Epoch {epoch:04d} | "
390                 f"Loss = {loss.item():.6e}"
391             )

```



## 19 Physics-Informed DeepONet in PyTorch

One of the major advantages of DeepONet is that the trunk network produces a differentiable function of the output coordinates.

Consider a one-dimensional PDE

$$\mathcal{L}[u](x) = f(x). \quad (80)$$

For example,

$$-\frac{d^2u}{dx^2} = f(x). \quad (81)$$

Let

$$u_\theta(x) = \mathcal{G}_\theta(a)(x). \quad (82)$$

Because the trunk network is differentiable, PyTorch automatic differentiation can calculate

$$\frac{\partial u_\theta}{\partial x} \quad (83)$$

and

$$\frac{\partial^2 u_\theta}{\partial x^2}. \quad (84)$$

The residual is

$$r_\theta(x) = -\frac{\partial^2 u_\theta}{\partial x^2} - f(x). \quad (85)$$

The physics loss is

$$\mathcal{L}_{\text{PDE}} = \frac{1}{N_c} \sum_{j=1}^{N_c} r_\theta(x_j)^2. \quad (86)$$

A corresponding PyTorch implementation is:

```
1
2 def pde_residual(
3     model,
4     branch_input,
5     x,
6     forcing
7 ):
8
9     x = x.clone().detach()
10    x.requires_grad_(True)
11
12    u = model(
13        branch_input,
14        x
15    )
16
17    # First derivative
18    du_dx = torch.autograd.grad(
19        u,
20        x,
21        grad_outputs=torch.ones_like(u),
```

```

22         create_graph=True
23     )[0]
24
25     # Second derivative
26     d2u_dx2 = torch.autograd.grad(
27         du_dx,
28         x,
29         grad_outputs=torch.ones_like(du_dx),
30         create_graph=True
31     )[0]
32
33     residual = (
34         -d2u_dx2
35         -
36         forcing
37     )
38
39     return residual

```

The total physics-informed loss can then be constructed as

```

1
2 residual = pde_residual(
3     model,
4     branch_input,
5     x_collocation,
6     forcing
7 )
8
9 loss_pde = torch.mean(
10     residual ** 2
11 )
12
13 loss = (
14     lambda_data * loss_data
15     +
16     lambda_pde * loss_pde
17 )

```

## 20 Summary

DeepONet learns a nonlinear operator using two complementary neural representations:

$$\boxed{\text{Branch Network} = \text{input-function representation}} \quad (87)$$

and

$$\boxed{\text{Trunk Network} = \text{output-coordinate representation.}} \quad (88)$$

Their interaction is

$$\boxed{\mathcal{G}_\theta(u)(y) = \sum_{k=1}^p \mathcal{B}_k(u(x_1), \dots, u(x_m)) \mathcal{T}_k(y)}. \quad (89)$$

The architecture can therefore be interpreted as a learned, input-dependent basis expansion of an output function.

The complete conceptual pipeline is

$$\boxed{u(\cdot) \xrightarrow{\text{sensorization}} [u(x_1), \dots, u(x_m)] \xrightarrow{\text{Branch}} \mathbf{b}(u)} \quad (90)$$

while simultaneously

$$\boxed{y \xrightarrow{\text{Trunk}} \mathbf{t}(y)}. \quad (91)$$

Finally,

$$\boxed{\hat{\mathcal{G}}(u)(y) = \mathbf{b}(u)^T \mathbf{t}(y)}. \quad (92)$$

This branch–trunk factorization is what distinguishes DeepONet from an ordinary neural network trained to map one finite-dimensional vector to another.

## References

- [1] L. Lu, P. Jin, G. Pang, Z. Zhang, and G. E. Karniadakis,  
*Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators*,  
 Nature Machine Intelligence, vol. 3, pp. 218–229, 2021.  
<https://doi.org/10.1038/s42256-021-00302-5>
- [2] L. Lu, X. Meng, S. Cai, Z. Mao, S. Goswami, Z. Zhang, and G. E. Karniadakis,  
*Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators*,  
 and subsequent physics-informed DeepONet formulations for parametric PDEs.